

5.00 credits

30.0 h + 15.0 h

Q2

Teacher(s)	Schaus Pierre ;
Language :	English > French-friendly
Place of the course	Louvain-la-Neuve
Main themes	• Constraints and domains • Practical aspects of constraint solvers • Constraint Satisfaction Problems (CSP) • Models and languages for constraint programming • Methods and techniques for constraint solving (consistency, relaxation, optimization, search, linear programming, global constraints, ...) • Search techniques and strategies • Problem modelling and resolution • Applications to different problem classes (e.g. planification, scheduling, ressource allocation, economics, robotics)
Learning outcomes	At the end of this learning unit, the student is able to : With regard to the AA reference framework of the "Master in Civil Engineering in Computer Science" program, this course contributes to the development, acquisition, and assessment of the following learning outcomes: • AA1.1. When faced with a computing problem, the student identifies the applicable concepts, algorithms, and data structures to solve it; they leverage these to break down the problem into subproblems and develop computational methods to solve them. • AA1.3. When confronted with results obtained through reasoning and the implementation of tools and concepts, the student takes the necessary step back to verify their relevance in terms of functionality and quality of the intended solution. In this context, they will develop relevant tests and verifications to ensure the quality of the developed solution. • AA2.4. During the implementation phase of the solution, the student demonstrates mastery of the principles, techniques, and development tools at their disposal. They create a software prototype to verify that it meets client expectations. They set up a testing environment and a suite of tests to ensure the developed solution meets the functional requirements. By applying validation and verification techniques, they identify, locate, and fix bugs. • AA5.4. The student is proficient in using reference books or manuals related to programming languages or software, both in English and French. They understand technical presentations given in English. • AA5.5. During the development of a software application, the student ensures traceability and documentation in a concise and precise manner, covering specifications, software and data structures, and operational procedures. They also apply this approach when writing a summary report explaining and justifying design and technology choices in project development. With regard to the AA reference framework of the "Master [120] in Computer Science" program, this course contributes to the development, acquisition, and assessment of the following learning outcomes: • AA1. Demonstrate mastery of a solid body of knowledge in computer science, enabling the resolution of problems within the discipline. • AA2.1. Analyze the problem to be solved or the functional requirements to be met and formulate the corresponding specifications. • AA2.2. Model the problem and design one or more original technical solutions that meet these specifications. • AA2.4. Implement and test the selected solution. • AA3.2. Propose a model and/or an experimental framework to simulate and test hypotheses related to the problem in all its complexity. • AA6.5. Self-evaluate and independently develop the knowledge necessary to remain competent in the field. • Understand a set of computer techniques for solving a constraint satisfaction problem. • Develop and extend a constraint programming solver. • Solve a simple application implementing the studied techniques. • Explain the foundations of constraint programming models and languages. • Identify classes of applications where constraint programming can be effectively used. • Model a simple problem as constraints and express this model in a constraint programming language, including the definition of search strategies.

Evaluation methods	June: For the first session, the overall course grade is based on: 30% assignments, 20% final project, 10% participation, and 40% exam. August: For the second session, only the exam can be retaken; the other grades remain unchanged. The August exam will be either written or oral. If deemed necessary by the instructor, an interview about any project may also be conducted also to verify that all theoretical concepts are well understood. Projects are individual. It means that any source code of a project estimated to be copied on the one of another student will result in a zero grade for the student at the projects and the exam. The same consequences will hold for a student that voluntarily shares his code or make available to other students. Nevertheless, students are permitted to use generative AI tools to assist with their assignments. Such tools can provide inspiration, suggest coding approaches, or help troubleshoot issues. But: • Original Work: While AI can be a tool, it should not be the sole author of your assignment. Your submission should be primarily your own work. Directly copying and pasting solutions from AI outputs without understanding or modification is discouraged. Similarly, collaborating with fellow students is a valuable part of the learning process, but directly copying another student's work is considered plagiarism. • Source Indication: Whenever you use generative AI to assist in your assignment, you are required to indicate so by providing a brief comment in your code on how the AI was used. For example: <pre># Used AI to suggest optimization for this loop. for i in range(10): ...</pre>
Teaching methods	The presentation of concepts will be delivered either through lectures, videos, or readings, and will be accompanied by practical work (assignments/micro-projects) requiring the application of these concepts.
Content	• Constraint Programming : a Declarative Programming paradigm • Architecture of a constraint programming solver • Global constraints and implementation techniques (incrementality, etc) • Search techniques and strategies • Combinatorial optimization problem modeling and solving • Modeling and solving combinatorial optimization problems using constraint programming • Applications to different problem classes (e.g. planification, scheduling, resource allocation, economics, robotics)
Inline resources	https://moodle.uclouvain.be/course/view.php?id=2009 www.minicp.org
Bibliography	Le site www.minicp.org + lectures suggérées pendant le semestre
Other infos	A good background in data-structure and algorithms is required to follow this course and a good knowledge of Java language
Faculty or entity in charge	INFO

Programmes containing this learning unit (UE)				
Program title	Acronym	Credits	Prerequisite	Learning outcomes
Master [120] in Computer Science and Engineering	INFO2M	5		
Master [120] in Computer Science	SINF2M	5		
Master [120] in Data Science Engineering	DATE2M	5		
Master [120] in Data Science: Information Technology	DAT12M	5		